Smart Dine — Restaurant Management System

Smart Dine is a comprehensive full-stack restaurant management system designed to streamline operations for restaurants, cafes, and dining establishments. It replaces legacy systems with a modern, scalable solution for point-of-sale (POS), kitchen management, inventory tracking, and customer engagement.

Features

- **User Management**: Role-based access control with roles like ADMIN, MANAGER, WAITER, KITCHEN, CASHIER
- **Table Management**: Interactive floor plan with drag-and-drop table positioning, QR code generation for customer ordering
- **Menu Management**: Digital menu with categories, items, pricing, and image uploads
- **Order Management**: Real-time order processing with WebSocket updates for live kitchen notifications
- **Kitchen Operations**: Order status tracking and preparation workflow
- **Billing & Payments**: Invoice generation with PDF support, payment processing
- **Inventory Management**: Stock tracking, supplier management, reorder alerts
- **Reporting**: Sales reports, insights, and analytics
- **Customer Portal**: QR-based self-service ordering for guests
- **Feedback System**: Customer satisfaction tracking
- **Employee Management**: Staff scheduling and attendance tracking

Tech Stack

Backend

- **Framework**: FastAPI (Python)
- **Database**: PostgreSQL (with SQLite fallback for development)
- **ORM**: SQLAlchemy 2.0
- **Authentication**: JWT with python-jose and passlib
- **WebSockets**: Real-time order updates
- **PDF Generation**: ReportLab
- **Task Scheduling**: APScheduler
- **Migration**: Alembic

Frontend

- **Framework**: React 18 with TypeScript
- **Build Tool**: Vite
- **Styling**: Tailwind CSS
- **Routing**: React Router
- **State Management**: React Context
- **Charts**: Recharts
- **Drag & Drop**: @dnd-kit
- **HTTP Client**: Axios
- **Notifications**: React Hot Toast
- **QR Codes**: qrcode.react

Architecture

Backend Architecture

The backend follows a modular FastAPI structure:

- `app/main.py`: Application entry point with CORS, static files, and router inclusion
- `app/models.py`: SQLAlchemy ORM models for database entities
- `app/database.py`: Database connection and session management
- `app/auth.py`: JWT authentication utilities
- `app/ws_manager.py`: WebSocket connection management
- `app/scheduler.py`: Background task scheduling
- `app/routers/`: API endpoint modules for different features
- `app/utils/`: Utility functions

Frontend Architecture

The frontend uses a component-based React structure:

- `src/App.tsx`: Main application with routing
- `src/components/`: Reusable UI components
- `src/pages/`: Page components for different views
- `src/contexts/`: React contexts for state management
- `src/hooks/`: Custom React hooks
- `src/api/`: API client and configuration
- `src/types/`: TypeScript type definitions
- `src/utils/`: Utility functions

Database Schema

The system uses the following main entities:

Core Entities

- **branches**: Restaurant branches/locations
- **users**: Staff users with role-based permissions
- **dining_tables**: Floor tables with QR tokens and positioning

- **categories**: Menu categories
- **menu_items**: Individual menu items with pricing and images

Order Management

- **orders**: Customer orders with status tracking
- **order_items**: Individual items within orders
- **payments**: Payment records for orders

Operations

- **inventory**: Stock items and quantities
- **suppliers**: Supplier information
- **purchases**: Purchase orders from suppliers
- **employees**: Staff information
- **attendance**: Employee attendance records
- **feedback**: Customer feedback on orders

Prerequisites

- Node.js 20+ and npm
- Python 3.11+
- PostgreSQL (recommended) — use a new empty database (do not reuse old databases; table names like `users` will conflict)

Installation & Setup

Backend Setup

```
cd backend
python -m venv .venv
# On Windows:
.venv\Scripts\activate
# On macOS/Linux:
# source .venv/bin/activate
pip install -r requirements.txt
```

Copy `.env.example` to `.env` and configure:

Variable	Example	Description
`DATABASE_URL`	`postgresql://user:pass@localhost:5432/smartdine`	Database connection string
`SECRET_KEY`	`your-long-random-secret-key-here`	JWT signing key
`CORS_ORIGINS`	`http://localhost:5173,http://127.0.0.1:5173`	Allowed frontend origins
`FRONTEND_PUBLIC_URL`	`http://localhost:5173`	Public URL for QR code links

Optional: Omit `DATABASE_URL` to use SQLite at `backend/smartdine.db`.

Initialize database:

```
python init_db.py
```

Seed demo data (creates admin user, sample branch, menu items, tables):

```
python scripts/seed_smart_dine.py
```

Default admin credentials after seeding: **`admin@smartdine.local`** / **`Admin@12345`**

Start the API server:

```
python run.py
# Or: uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

API will be available at `http://127.0.0.1:8000` with docs at `/docs`.

Frontend Setup

```
cd frontend
npm install
```

Create `.env` file:

```
VITE_BACKEND_URL=http://127.0.0.1:8000
```

Start development server:

```
npm run dev
```

Frontend will be available at `http://localhost:5173`.

API Overview

The REST API provides endpoints for all system operations:

Authentication

- `POST /api/auth/login` - User login
- `POST /api/auth/register` - User registration
- `GET /api/auth/me` - Get current user info

Core Operations

- `/api/branches` - Branch management
- `/api/tables` - Table management
- `/api/menu` - Menu and categories
- `/api/orders` - Order management
- `/api/kitchen` - Kitchen operations
- `/api/billing` - Billing and payments
- `/api/inventory` - Inventory tracking
- `/api/suppliers` - Supplier management
- `/api/employees` - Employee management
- `/api/reports` - Reporting endpoints
- `/api/insights` - Analytics and recommendations

Customer Portal

- `/api/public-customer/menu` - Public menu access
- `/api/public-customer/orders` - Customer order placement

Real-time Updates

- `WebSocket /ws/orders` - Live order status updates

Full API documentation available at `/docs` when the backend is running.

User Roles & Permissions

- **ADMIN**: Full system access
- **MANAGER**: Branch management, reports, employee oversight
- **WAITER**: Table management, order taking
- **KITCHEN**: Order preparation status updates
- **CASHIER**: Billing and payment processing

Customer Experience

Customers can scan QR codes on tables to access a self-service ordering interface at `/customer/{qr_token}`, allowing them to:

- Browse the digital menu
- Place orders directly
- View order status
- Provide feedback

WebSocket Integration

The system uses WebSockets for real-time updates:

- Order creation/modification broadcasts to kitchen and waiter interfaces
- Status changes trigger immediate UI updates
- Connection: `ws://<host>:8000/ws/orders`

Additional Features

- **Insights**: Basic sales predictions and recommendations (extensible for ML integration)
- **PWA Ready**: Manifest file included for progressive web app capabilities
- **PDF Invoices**: Automatic invoice generation for orders
- **File Uploads**: Menu item images and other assets
- **Email/SMS Ready**: Infrastructure for notifications (requires provider integration)

Development

Running Tests

```
# Backend tests (if implemented)
cd backend
pytest

# Frontend linting
cd frontend
npm run lint
```

Building for Production

```
# Frontend build
cd frontend
npm run build

# Backend can be deployed with uvicorn or ASGI server
```

Database Migrations

```
cd backend
alembic revision --autogenerate -m "Migration message"
alembic upgrade head
```

Troubleshooting

Common Issues

1. **Database Connection Errors**
    - Ensure PostgreSQL is running
    - Verify `DATABASE_URL` in `.env`
    - Check database user permissions
2. **CORS Errors**
    - Verify `CORS_ORIGINS` includes your frontend URL
    - Check for trailing slashes in URLs
3. **WebSocket Connection Issues**
    - Ensure backend is running on the correct host/port
    - Check firewall settings for WebSocket connections
4. **File Upload Issues**
    - Verify `uploads/` directory exists and is writable
    - Check file size limits in your deployment

Logs

- Backend logs to console with logging level INFO
- Frontend errors visible in browser console
- Check `/api/health` endpoint for backend status

Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests if applicable
5. Submit a pull request

Documentation

- **API Reference (docs/API.md)** - Complete API endpoint documentation
- **Development Guide (docs/DEVELOPMENT.md)** - Codebase structure and development workflow
- **Deployment Guide (docs/DEPLOYMENT.md)** - Production deployment instructions